

Day 18 – React Introduction and Setup (Doctors App)

Topics Covered

1. What is React
 2. SPA Concept
 3. Project Setup
 4. Project Structure
 5. JSX
 6. Functional Components
-

1. What is React?

React is a JavaScript library used to build User Interfaces.

UI means:

- Navbar
 - Cards
 - Forms
 - Buttons
 - Tables
 - Complete websites
-

Why React?

Suppose we have:

- 100 doctors

Without React:

- We repeat same HTML again and again.

With React:

- Create one DoctorCard component
- Reuse it multiple times

This saves:

- Time
- Code

- Effort
-

Real-Time Example

Instead of writing:

HTML

```
<div>
  <h1>Dr.Teja</h1>
</div>

<div>
  <h1>Dr.Sam</h1>
</div>
```

We create:

None

```
<DoctorCard />
```

and reuse it.

2. SPA Concept

SPA means: **Single Page Application**

Traditional Websites

In old websites:

- Every click reloads complete page.

Example:

- Click Home
- Full page reload

Problems:

- Slow
- Bad user experience

React Applications

In React:

- Page loads once
- Only content changes

Example:

- Home Page
- Add Doctor Page

Only component changes.

Entire page does NOT reload.

This makes applications:

- Faster
 - Smooth
 - Modern
-

3. React Project Setup

We use:

- Node.js
 - Vite
 - React
-

Step 1 – Open VS Code

Open terminal.

Step 2 – Create React Project

Shell

```
npm create vite@latest
```

Step 3 – Enter Project Name

None

`doctors-app`

Step 4 – Select Framework

Choose:

None

`React`

Step 5 – Select Variant

Choose:

None

`JavaScript`

Step 6 – Move into Project

Shell

`cd doctors-app`

Step 7 – Install Packages

Shell

`npm install`

Step 8 – Run Project

Shell

`npm run dev`

Output

None

`http://localhost:5173`

Open in browser.

4. Project Structure

After project creation:

Shell

```
doctors-app
├── node_modules
├── public
├── src
└── package.json
```

Important Folder – src

Most React code is written inside:

None

`src`

Important Files

Shell

```
src
├── App.jsx
├── main.jsx
└── index.css
```

App.jsx

Main component.

Our Doctors App starts here.

main.jsx

Application entry point.

index.css

Global CSS file.

Folder Structure We Will Follow

Create folders:

Shell

src

|

|— components

|— pages

|— App.jsx

Components Folder

Reusable UI:

- Navbar
 - DoctorCard
-

Pages Folder

Separate pages:

- Home.jsx
 - AddDoctor.jsx
-

5. JSX

JSX means:

None

JavaScript XML

JSX allows us to write:

- HTML inside JavaScript
-

JSX Example

None

```
function App() {  
  
  return (  
    <div>  
  
      <h1>Doctors App</h1>  
  
    </div>  
  )  
}  
  
export default App
```

Why JSX?

Without JSX:

JavaScript

React.createElement()

Difficult for beginners.

With JSX:

- Easy
 - Clean
 - Readable
-

JSX Rules

Rule 1 – Single Parent Element

Wrong:

None

```
return (  
  <h1>Hello</h1>  
  <p>React</p>  
)
```

Correct:

None

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>React</p>  
  </div>  
)
```

Rule 2 – class becomes className

HTML:

HTML

```
<div class="card"></div>
```

JSX:

None

```
<div className="card"></div>
```

Rule 3 – JavaScript inside {}

None

```
const name = "Teja"  
  
<h1>{name}</h1>
```

6. Functional Components

Components are reusable UI blocks.

Examples:

- Navbar
 - Doctor Card
 - Footer
-

Why Components?

Suppose we have:

- 100 doctors

Instead of writing same HTML:

- Create one reusable component.
-

Create Navbar Component

Inside:

None

`src/components`

Create file:

None

`Navbar.jsx`

Navbar.jsx

```
function Navbar() {  
  
  return (  
  
    <div className="navbar">  
  
      <h1>Doctors App</h1>  
  
      <div className="nav-links">  
        <button className="btn-1">Home</button>  
      </div>  
    </div>  
  )  
}
```

```
        <button className="btn-2">Add Doctor</button>
      </div>

    </div>
  )
}

export default Navbar
```

App.css

```
*{
  margin: 0px;
  padding: 0px;
  box-sizing: border-box;
}

.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px 20px;
  background: black;
  color: white;
}

.nav-links {
  display: flex;
  gap: 10px;
}

.btn-1, .btn-2 {
  padding: 8px 35px;
  cursor: pointer;
  border: 2px solid white;
  color: white;
  background: transparent;
  border-radius: 8px;
}
```

Create DoctorCard Component

Create:

None

DoctorCard.jsx

DoctorCard.jsx

```
function DoctorCard() {

  return (

    <div className="card">

      <div className="card-body">

        <div>
          <h2>Dr.Teja</h2>
          <p className="subtitle">Muscles Specialist</p>
          <p className="subtitle">Male</p>
        </div>
      </div>
      <button className="btn-2">View Details</button>
    </div>
  )
}

export default DoctorCard
```

CSS for DoctorCard.jsx:

Add this is App.css

```
.card{
  width: 400px;
  padding: 20px;
  border: 1px solid black;
  border-radius: 8px;
```

```

    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  }
  .card-body{
    display: flex;
    gap: 20px;
    align-items: center;
  }
  .subtitle{
    color: gray;
  }
  .btn-2{
    padding: 8px 35px;
    cursor: pointer;
    background: transparent;
    border-radius: 8px;
    color: black;
    border: 2px solid black;
    width: 100%;
    margin-top: 20px;
  }

```

Using Components in App.jsx

```

import Navbar from "../components/Navbar"
import DoctorCard from "../components/DoctorCard"
import './App.css'

function App() {

  return (

    <div>

      <Navbar />

      <h1>All Doctors</h1>

      <div className="container">
        <DoctorCard />
        <DoctorCard />
        <DoctorCard />
        <DoctorCard />
      </div>
    </div>
  )
}

```

```

    </div>

  </div>
)
}

export default App

```

CSS:

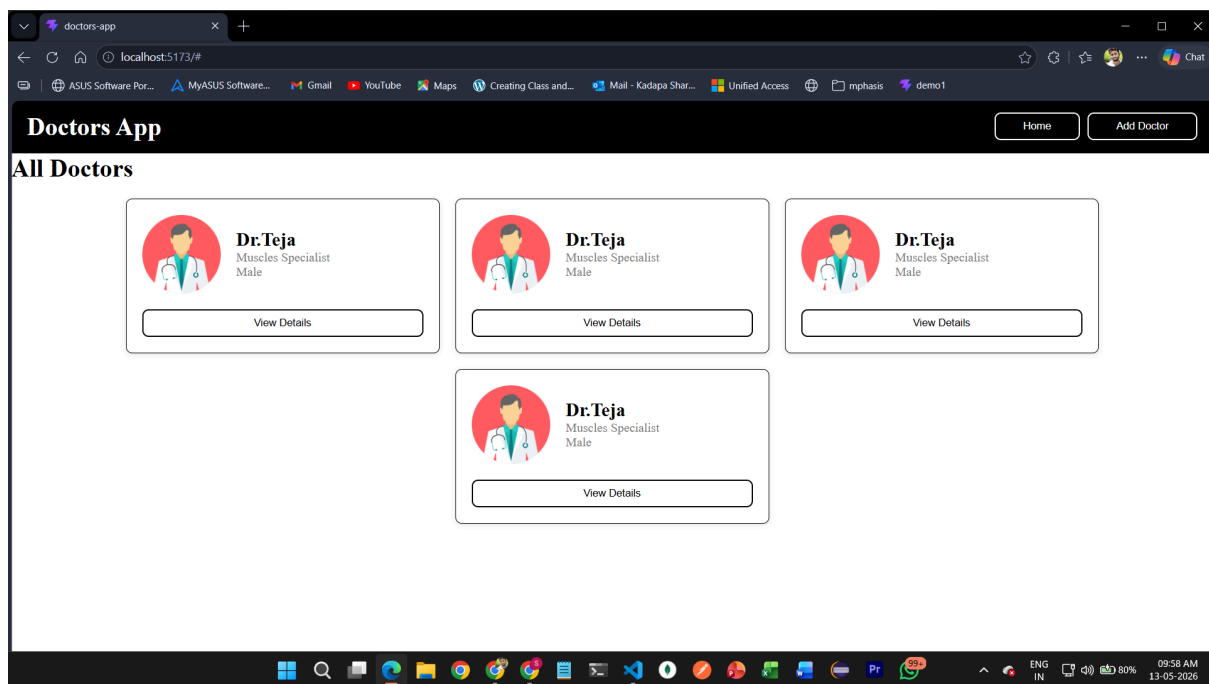
Update App.css

```

.container{
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
  justify-content: center;
  margin-top: 20px;
}

```

Output

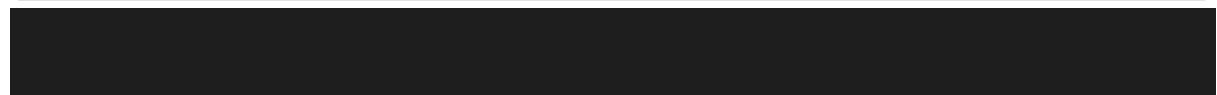


End of Today Class

Today students completed:

- React setup
- Project structure
- JSX
- Functional components
- Navbar component
- DoctorCard component
- Basic Doctors App UI

Day 19 – React Hooks and Forms



1. Props in React

What are Props?

Props means: **Properties**

Props are used to:

- Pass data
 - Parent Component → Child Component
-

Real-Time Understanding

Suppose we have:

- 100 doctors

Without props:

- We must create 100 separate cards manually.

With props:

- Create one DoctorCard component
- Reuse it with different data

This is one of the biggest advantages of React.

Example With Props

None

```
<DoctorCard
  name="Teja"
  specialization="Muscles"
/>

<DoctorCard
  name="Sam"
  specialization="Bones"
/>
```

Reusable component.

Parent Component

Create Home.jsx component

Home.jsx

```
import DoctorCard from "../components/DoctorCard"

function Home() {

  return (

    <div className="container">

      <DoctorCard
        name="Dr. Sam"
        gender="Male"
        specialization="Muscles"
      />

      <DoctorCard
        name="Dr. Priya"
        gender="Female"
        specialization="Cardiology"
      />

      <DoctorCard
        name="Dr. Ahmed"
        gender="Male"
        specialization="Neurology"
      />

    </div>

  )
}
```

```

        <DoctorCard
          name="Dr. Sneha"
          gender="Female"
          specialization="Orthopedics"
        />

        <DoctorCard
          name="Dr. Arjun Reddy"
          gender="Male"
          specialization="ENT"
        />

        <DoctorCard
          name="Dr. Kavya"
          gender="Female"
          specialization="Gynecology"
        />
      </div>
    )
  }

export default Home

```

Child Component DoctorCard.jsx

```

function DoctorCard(props) {

  return (

    <div className="card">

      <div className="card-body">
        
        <div>
          <h2>Dr. {props.name}</h2>

```



```

        <p className="subtitle">{props.specialization}
Specialist</p>
        <p className="subtitle">{props.gender}</p>
      </div>
    </div>
    <button className="btn-1">View Details</button>

  </div>
)
}

export default DoctorCard

```

Understanding Flow

```

None
Home Component
  ↓
Passing Props
  ↓
DoctorCard Component

```

Destructuring Props

Instead of:

```

None
props.name

```

We can write:

```

None
function DoctorCard({
  name,
  gender,
  specialization
})

```

Updated DoctorCard.jsx

```
function DoctorCard({ name, gender, specialization }) {

  return (

    <div className="card">

      <div className="card-body">
        
        <div>
          <h2>Dr.{name}</h2>
          <p className="subtitle">{specialization}
Specialist</p>
          <p className="subtitle">{gender}</p>
        </div>
      </div>
      <button className="btn-1">View Details</button>

    </div>
  )
}

export default DoctorCard
```

2. useState Hook

Now we want:

- User input
- Dynamic form values
- Add doctor data

For this we use:

None

useState

What is State?

State means:

- Dynamic data
- Data that changes

Examples:

- Input values
 - Login status
 - Counter value
 - API response
-

Import useState

None

```
import { useState } from "react"
```

useState Syntax

None

```
const [state, setState] = useState(initialValue)
```

Understanding Syntax

Part	Meaning
state	Current value
setState	Update function
initialValue	Default value

Example

None

```
const [name, setName] = useState("")
```

Initially:

None

```
name = ""
```

After typing:

- State updates automatically.
-

Create Form States

AddDoctor.jsx

None

```
const [name, setName] = useState("")
const [age, setAge] = useState("")
const [gender, setGender] = useState("")
const [specialization, setSpecialization] = useState("")
const [salary, setSalary] = useState("")
```

Why Multiple States?

Because every input needs separate value.

Example:

- Name input → name state
 - Age input → age state
-

3. Controlled Components

What is Controlled Component?

Controlled Component means:

- Input controlled by React state
-

Why Important?

Without controlled components:

- React cannot manage form properly.

With controlled components:

- React fully controls form data.
-

Example

None

```
<input
  type="text"
  value={name}
  onChange={(e)=>setName(e.target.value)}
/>
```

Understanding Input value={name}

Displays state value inside input.

onChange

Runs whenever user types.

Understanding Event

None

```
(e)=>setName(e.target.value)
```

Part	Meaning
e	Event object
e.target	Current input
e.target.value	Typed value

4. Form Handling

Now we will create:

- Add Doctor Form
-

Create AddDoctor Page

pages/AddDoctor.jsx

```
import { useState } from "react"

function AddDoctor() {

  const [name, setName] = useState("")
  const [age, setAge] = useState("")
  const [gender, setGender] = useState("")
  const [specialization, setSpecialization] = useState("")
  const [salary, setSalary] = useState("")

  return (

    <div className="">

      <h1 style={{ textAlign: "center"}}>Add New Doctor</h1>

      <form className="form">

        <input
          type="text"
          placeholder="Enter Doctor Name"
          value={name}
          onChange={ (e) => setName(e.target.value) }
          className="text-box"
        />

        <input
          type="number"
          placeholder="Enter Age"
          value={age}
          onChange={ (e) => setAge(e.target.value) }
          className="text-box"
        />

        <select
          value={gender}
          onChange={ (e) => setGender(e.target.value) }
          className="text-box"
        >
```

```

        <option value="">
            Select Gender
        </option>

        <option value="Male">
            Male
        </option>

        <option value="Female">
            Female
        </option>

    </select>

    <input
        type="text"
        placeholder="Enter Specialization"
        value={specialization}
        onChange={ (e) => setSpecialization(e.target.value) }
        className="text-box"
    />

    <input
        type="number"
        placeholder="Enter Salary"
        value={salary}
        onChange={ (e) => setSalary(e.target.value) }
        className="text-box"
    />

    <button className="btn-2" style={{ backgroundColor:
"black", color: "white" }}>
        Add Doctor
    </button>

</form>

</div>

)
}

```

```
export default AddDoctor
```

CSS:

```
.text-box{
  width: 100%;
  padding: 10px;
  border: 1px solid black;
  border-radius: 5px;
  margin-bottom: 20px;
}

.form{
  width: 400px;
  padding: 30px;
  border: 1px solid black;
  border-radius: 8px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  margin: 20px auto;
}
```

Output

Add New Doctor

Select Gender

▼

Add Doctor

Students can:

- Type values
 - React stores values inside state
-

5. Event Handling

What are Events?

Events means:

- User actions

Examples:

- Click
 - Submit
 - Typing
-

onClick Event

None

```
<button onClick={handleClick}>
```

Runs when button clicked.

onChange Event

None

```
onChange={(e)=>setName(e.target.value)}
```

Runs when typing.

onSubmit Event

None

```
<form onSubmit={handleSubmit}>
```

Runs when form submitted.

Create handleSubmit Function

AddDoctor.jsx

```
function handleSubmit(e) {  
  e.preventDefault();  
  const newDoctor = {  
    name,  
    age,  
    gender,  
    specialization,  
    salary  
  };  
  console.log(newDoctor);  
  
  setName("")  
  setAge("")  
  setGender("")  
  setSpecialization("")  
  setSalary("")  
}
```

Understanding preventDefault()

Normally:

- Form reloads page after submit.

None

`e.preventDefault()`

Stops page reload.

Understanding Form Object

None

```
const newDoctor = {  
  name,  
  age,  
  gender,  
  specialization,  
  salary
```

```
}
```

Stores complete doctor information.

Connect Form

None

```
<form onSubmit={handleSubmit}>
```

Updated AddDoctor.jsx

```
import { useState } from "react"

function AddDoctor() {

  const [name, setName] = useState("")
  const [age, setAge] = useState("")
  const [gender, setGender] = useState("")
  const [specialization, setSpecialization] = useState("")
  const [salary, setSalary] = useState("")

  function handleSubmit(e) {
    e.preventDefault();
    const newDoctor = {
      name,
      age,
      gender,
      specialization,
      salary
    };
    console.log(newDoctor);

    setName("")
    setAge("")
    setGender("")
    setSpecialization("")
    setSalary("")
  }

  return (
```



```

        </select>

        <input
            type="text"
            placeholder="Enter Specialization"
            value={specialization}
            onChange={ (e) => setSpecialization(e.target.value) }
            className="text-box"
        />

        <input
            type="number"
            placeholder="Enter Salary"
            value={salary}
            onChange={ (e) => setSalary(e.target.value) }
            className="text-box"
        />

        <button className="btn-2" style={{ backgroundColor:
"black", color: "white" }}>
            Add Doctor
        </button>

    </form>

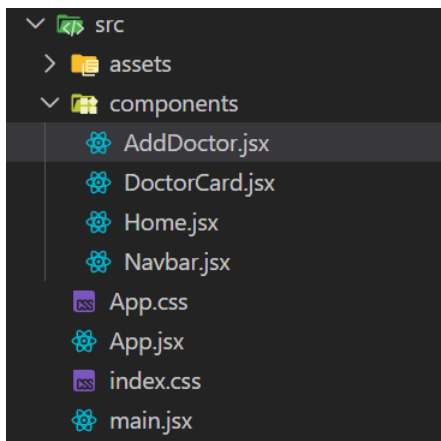
</div>

)
}

export default AddDoctor

```

Folder Structure



End of Today Class

Students completed:

- Props
 - useState
 - Controlled Components
 - Form Handling
 - Event Handling
 - Add Doctor Page
 - Dynamic Doctor Cards
-

Day 20 – Lists and Conditional Rendering

Topics Covered

1. useEffect
 2. map()
 3. key
 4. Ternary Operator
 5. Conditional Rendering
-

Step 1 – Create Static Doctors Data Home.jsx

Understanding Data Structure

Doctors data is:

- Array of objects
-

Understanding Single Object

JavaScript

```
{  
  id:1,  
  name:"Teja",  
  age:26  
}
```

Why Array?

Because:

- Multiple doctors
-

Why Object?

Because one doctor has:

- Name
 - Age
 - Gender
 - Salary
-

1. useEffect Hook

What is useEffect?

useEffect is used for:

- Side effects

Examples:

- API calls
 - Timers
 - Local storage
-

Today Example

We will use useEffect:

- To understand component loading
-

Import useEffect

None

```
import { useEffect } from "react"
```

Example

None

```
useEffect(() => {  
  console.log("Home Page Loaded")  
}, [])
```

Understanding []

None

[]

Means:

- Run only once
 - After component load
-

Real-Time Understanding

None

Home Page Opens



useEffect Runs



Console Message Displays

2. map() Method

What is map()?

map() is used to:

- Loop through arrays
 - Display data dynamically
-

Why map()?

Suppose:

- 100 doctors

Without map():

- Write 100 cards manually

With map():

- React automatically creates cards
-

map() Syntax

None

```
array.map((item) => {  
  
}))
```

Example

None

```
{  
  doctors.map((doctor) => (  
  
    <h1>{doctor.name}</h1>  
  
  ))  
}
```

Output

None

```
Teja  
Sam  
Anu
```

Create Doctor Cards Dynamically

Home.jsx

```
import { useState ,useEffect} from "react"  
import DoctorCard from "../components/DoctorCard"  
  
function Home() {  
  const[doctors,setDoctors] = useState([]);  
  
  function fetchDoctors() {  
    // use static data for now, later we will fetch data from  
server  
    let data = [  
      {  
        id: 1,  
        name: "Teja",  
        age: 26,  
        gender: "Male",
```

```

        specialization: "Muscles",
        salary: 7000000
    },

    {
        id: 2,
        name: "Sam",
        age: 26,
        gender: "Male",
        specialization: "Bones",
        salary: 4000000
    },

    {
        id: 3,
        name: "Anu",
        age: 25,
        gender: "Female",
        specialization: "Heart",
        salary: 5000000
    }
]
setDoctors(data);
}

let result = doctors.map((doctor) => {
    return (
        <DoctorCard
            key={doctor.id}
            name={doctor.name}
            specialization={doctor.specialization}
        />
    )
})

useEffect(() => {
    fetchDoctors();
}, [])

return (
    <div className="container">

```

```

        {
            doctors.length > 0 ? result : <h1>No Doctors
Found</h1>
        }
    }

    </div>
)
}

export default Home

```

3. key Property

Why key is Important?

When rendering lists:

- React needs unique identification

Without key:

- Warning appears in console

Example

None

```
key={doctor.id}
```

Updated map()

None

```

doctors.map((doctor) => (
    <DoctorCard
      key={doctor.id}
      name={doctor.name}
      gender={doctor.gender}
      specialization={doctor.specialization}
    />

```

```
))
```

Why Use ID?

Because:

- ID is unique

Best practice:

- Always use unique database ID
-

4. Conditional Rendering

What is Conditional Rendering?

Showing UI based on condition.

Examples:

- Loading...
 - No Data
 - Success Message
-

5. Ternary Operator

Syntax

None

```
condition ? trueBlock : falseBlock
```

Example

None

```
{
  doctors.length > 0

  ?

  <h1>Doctors Available</h1>

  :
```

```
<h1>No Doctors Found</h1>
}
```

Real-Time Understanding

If:

- doctors available

Show:

- Doctor cards

Else:

- Empty message
-

What Students Learned Today

Students completed:

- useEffect
 - map()
 - key
 - Conditional Rendering
 - Ternary Operator
 - Dynamic doctor cards
-

Day 21 – Search and Deployment (Doctors App)

Topics Covered

1. filter()
 2. map()
 3. Search Input Handling
 4. Conditional Rendering
 5. GitHub Deployment
 6. Vercel Deployment
-

Current Problem

Currently:

- All doctors display together.

But users want:

- Search doctor by name
- Filter doctor by specialization

Example:

- Search "Teja"
 - Show only Teja
-

Static Doctors Data

Home.jsx

None

```
const doctors = [  
  
  {  
    id:1,  
    name:"Teja",  
    age:26,  
    gender:"Male",  
    specialization:"Muscles",
```

```
    salary:7000000
  },
  {
    id:2,
    name:"Sam",
    age:26,
    gender:"Male",
    specialization:"Bones",
    salary:4000000
  },
  {
    id:3,
    name:"Anu",
    age:25,
    gender:"Female",
    specialization:"Heart",
    salary:5000000
  }
]
```

1. filter() Method

What is filter()?

filter() is used to:

- Filter array data
- Return matching items

Syntax

JavaScript

```
array.filter((item) => {  
  
  })
```

Real-Time Example

Suppose:

None

Teja

Sam

Anu

Search:

None

Te

Result:

None

Teja

Create Search State

Home.jsx

None

```
const [search, setSearch] = useState("")
```

Why useState?

Because:

- Search value changes dynamically

Create Search Input

None

```
<input  
  type="text"  
  placeholder="Search Doctor"  
  value={search}  
  onChange={(e)=>setSearch(e.target.value)}  
>
```

Understanding onChange

Whenever user types:

- State updates automatically
-

Create Filtered Doctors

None

```
const filteredDoctors = doctors.filter((doctor) => {  
  
  return doctor.name  
    .toLowerCase()  
    .includes(search.toLowerCase())  
  
})
```

Understanding Logic

toLowerCase()

Converts text into lowercase.

includes()

Checks matching text.

Example

Doctor name:

None

Teja

Search:

None

te

Still works because:

- Both converted into lowercase
-

Display Filtered Doctors

Replace:

```
None  
doctors.map()
```

with:

```
None  
filteredDoctors.map()
```

Updated Display

```
None  
<div className="doctors-container">  
  
  {  
    filteredDoctors.map((doctor) => (  
  
      <DoctorCard  
        key={doctor.id}  
        name={doctor.name}  
        age={doctor.age}  
        gender={doctor.gender}  
        specialization={doctor.specialization}  
        salary={doctor.salary}  
      />  
  
    ))  
  }  
  
</div>
```

Output

Students can now:

- Search doctors dynamically
-

2. Filter by Specialization

Now we will add:

- Dropdown filter
-

Create Filter State

None

```
const [specialization, setSpecialization] = useState("")
```

Create Select Dropdown

None

```
<select
  value={specialization}
  onChange={(e)=>setSpecialization(e.target.value)}
>

  <option value="">
    All
  </option>

  <option value="Muscles">
    Muscles
  </option>

  <option value="Bones">
    Bones
  </option>

  <option value="Heart">
    Heart
  </option>

</select>
```

Update filter()

None

```
const filteredDoctors = doctors.filter((doctor) => {  
  
  const matchesSearch = doctor.name  
    .toLowerCase()  
    .includes(search.toLowerCase())  
  
  const matchesSpecialization = specialization === ""  
  
  ||  
  
  doctor.specialization === specialization  
  
  return matchesSearch && matchesSpecialization  
})
```

Understanding matchesSearch

Checks:

- Doctor name matching
-

Understanding matchesSpecialization

Checks:

- Selected specialization matching
-

Final Return

None

```
return matchesSearch && matchesSpecialization
```

Means:

- Both conditions must be true
-

3. map() with Filtered Data

Now map() displays:

- Only filtered doctors
-

Example

Search:

None
Te

Output:

None
Teja only

4. Conditional Rendering

Now we will show:

- No Doctors Found

when search result is empty.

Ternary Operator
Syntax

None
`condition ? trueBlock : falseBlock`

Example

None

```
{
  filteredDoctors.length > 0

  ?

  <h1>Doctors Available</h1>

  :
```

```
<h1>No Doctors Found</h1>
}
```

Real-Time Example

Search:

None
XYZ

No doctors available.

Show:

None
No Doctors Found

Complete Home.jsx

```
None
import { useEffect, useState } from "react"

import DoctorCard from "../components/DoctorCard"

function Home() {

  const [search, setSearch] = useState("")

  const [specialization, setSpecialization] = useState("")

  const doctors = [

    {
      id:1,
      name:"Teja",
      age:26,
      gender:"Male",
      specialization:"Muscles",
      salary:7000000
    },
  ],
```

```

    {
      id:2,
      name:"Sam",
      age:26,
      gender:"Male",
      specialization:"Bones",
      salary:4000000
    },

    {
      id:3,
      name:"Anu",
      age:25,
      gender:"Female",
      specialization:"Heart",
      salary:5000000
    }
  ]

  useEffect(() => {

    console.log("Home Page Loaded")

  }, [])

  const filteredDoctors = doctors.filter((doctor) => {

    const matchesSearch = doctor.name
      .toLowerCase()
      .includes(search.toLowerCase())

    const matchesSpecialization = specialization === ""

    ||

    doctor.specialization === specialization

    return matchesSearch && matchesSpecialization
  })

  return (

    <div className="container">

```



```
<h1>All Doctors</h1>

<div className="filters">

  <input
    type="text"
    placeholder="Search Doctor"
    value={search}
    onChange={(e)=>setSearch(e.target.value)}
  />

  <select
    value={specialization}
    onChange={(e)=>setSpecialization(e.target.value)}
  >

    <option value="">
      All
    </option>

    <option value="Muscles">
      Muscles
    </option>

    <option value="Bones">
      Bones
    </option>

    <option value="Heart">
      Heart
    </option>

  </select>

</div>

<br />

{
  filteredDoctors.length > 0

  ?

  <div className="doctors-container">
```

```

        {
          filteredDoctors.map((doctor) => (
            <DoctorCard
              key={doctor.id}
              name={doctor.name}
              age={doctor.age}
              gender={doctor.gender}
              specialization={doctor.specialization}
              salary={doctor.salary}
            />
          ))
        }
      </div>

      :
      <h1>No Doctors Found</h1>
    }
  </div>
)
}

export default Home

```

Basic Styling App.css

```

CSS

.container{
  padding:20px;
}

.filters{
  display:flex;
  gap:20px;
}

input,

```

```
select{
  padding:10px;
  width:250px;
}

.doctors-container{
  display:flex;
  gap:20px;
  flex-wrap:wrap;
}

.card{
  width:250px;
  border:1px solid gray;
  padding:20px;
  background:white;
}
```

5. GitHub Deployment

Step 1 – Initialize Git

Shell

```
git init
```

Step 2 – Add Files

Shell

```
git add .
```

Step 3 – Commit

Shell

```
git commit -m "Doctors App"
```

Step 4 – Create Repository

Create:

- doctors-app repository
-

Step 5 – Connect Repository

Shell

```
git remote add origin YOUR_GITHUB_LINK
```

Step 6 – Push Code

Shell

```
git push -u origin main
```

6. Vercel Deployment

Go to:

[Vercel](#)

Steps

1. Login using GitHub
 2. Click:
 - Add New Project
 3. Import:
 - doctors-app repository
 4. Click:
 - Deploy
-

Output

Students get:

- Live application link

Example:

None

```
https://doctors-app.vercel.app
```

What Students Learned Today

Students completed:

- Search functionality
- Filter functionality
- `filter()`
- `map()`
- Conditional rendering
- GitHub deployment
- Vercel deployment

And improved:

None

Doctors App

Day 22 – Routing (Doctors App)

Topics Covered

1. React Router Setup
 2. Routes
 3. useNavigate
 4. useParams
-

What is Routing?

Routing means:

- Switching between pages

Examples:

- Home Page
- About Page
- Contact Page

In React:

- Page does NOT reload completely

Only component changes.

Why Routing?

Suppose user clicks:

None

[View Details](#)

We should navigate to:

None

</doctor/2>

and display:

- Complete doctor details

For this we use:

None

React Router

1. Install React Router

Open terminal:

Shell

```
npm install react-router-dom
```

2. Setup BrowserRouter

main.jsx

None

```
import React from 'react'
import ReactDOM from 'react-dom/client'

import { BrowserRouter } from "react-router-dom"

import App from './App'

import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(

  <BrowserRouter>

    <App />

  </BrowserRouter>

)
```

What is BrowserRouter?

BrowserRouter enables:

- Routing in React application

Without BrowserRouter:

- Routes will not work
-

3. Create Routes

App.jsx

None

```
import Navbar from "../components/Navbar"

import Home from "../pages/Home"
import AddDoctor from "../pages/AddDoctor"

import {
  Routes,
  Route
} from "react-router-dom"

function App() {

  return (

    <div>

      <Navbar />

      <Routes>

        <Route
          path="/"
          element={<Home />}
        />

        <Route
          path="/add-doctor"
          element={<AddDoctor />}
        />

      </Routes>

    </div>
  )
}

export default App
```

Understanding Routes

Route	Page
/	Home Page
/add-doctor	Add Doctor Page

4. Navbar Navigation

Currently buttons do nothing.

Now we will navigate pages.

useNavigate Hook

useNavigate is used for:

- Page navigation
-

Import useNavigate
Navbar.jsx

None

```
import { useNavigate } from "react-router-dom"
```

Create navigate Variable

None

```
const navigate = useNavigate()
```

Update Navbar.jsx

None

```
import { useNavigate } from "react-router-dom"

function Navbar() {

  const navigate = useNavigate()
```

```
return (  
  <div className="navbar">  
    <h1>Doctors App</h1>  
    <div>  
      <button onClick={() => navigate("/")}>  
        Home  
      </button>  
  
      <button  
        onClick={() => navigate("/add-doctor")}  
      >  
        Add Doctor  
      </button>  
    </div>  
  </div>  
)  
}  
  
export default Navbar
```

Understanding navigate()

None

`navigate("/")`

Moves user to:

- Home Page
-

Output

Students can now:

- Switch pages dynamically

Without:

- Page reload
-

5. Create Doctor Details Page

Now we will create:

- Separate doctor details page

When user clicks:

None

[View Details](#)

Navigate to:

None

</doctor/2>

Create New Page

pages/DoctorDetails.jsx

None

```
function DoctorDetails() {  
  return (  
    <div className="container">  
      <h1>Doctor Details Page</h1>  
    </div>  
  )  
}  
  
export default DoctorDetails
```

Add Route

App.jsx

None

```
import DoctorDetails from "../pages/DoctorDetails"
```

Add Route

None

```
<Route  
  path="/doctor/:id"  
  element={<DoctorDetails />}  
</>
```

Understanding :id

None

```
/doctor/:id
```

Means:

- Dynamic route parameter

Examples:

None

```
/doctor/1  
/doctor/2  
/doctor/10
```

6. useNavigate in DoctorCard

Now when user clicks:

- View Details

Navigate to:

- Doctor Details Page
-

Update DoctorCard.jsx

None

```
import { useNavigate } from "react-router-dom"

function DoctorCard({
  id,
  name,
  age,
  gender,
  specialization,
  salary
}) {

  const navigate = useNavigate()

  return (

    <div className="card">

      <h2>{name}</h2>

      <p>{specialization}</p>

      <p>Age: {age}</p>

      <p>Gender: {gender}</p>

      <p>Salary: {salary}</p>

      <button
        onClick={() => navigate(`/doctor/${id}`)}
      >
        View Details
      </button>

    </div>
  )
}

export default DoctorCard
```

Important Understanding

None

```
navigate(`/doctor/${id}`)
```

Example:

None

```
/doctor/2
```

Pass id from Home.jsx

None

```
<DoctorCard  
  key={doctor.id}  
  id={doctor.id}  
  name={doctor.name}  
  age={doctor.age}  
  gender={doctor.gender}  
  specialization={doctor.specialization}  
  salary={doctor.salary}  
>
```

7. useParams Hook

Now we need:

- Route ID

Example:

None

```
/doctor/2
```

Need:

None

```
2
```

For this we use:

None

`useParams`

Import useParams
DoctorDetails.jsx

None

`import { useParams } from "react-router-dom"`

Get Route Parameter

None

`const { id } = useParams()`

Understanding

If URL is:

None

`/doctor/5`

Then:

None

`id = 5`

Display ID
DoctorDetails.jsx

None

`import { useParams } from "react-router-dom"`

`function DoctorDetails() {`

`const { id } = useParams()`

`return (`

```
    <div className="container">

      <h1>Doctor Details Page</h1>

      <h2>Doctor ID: {id}</h2>

    </div>
  )
}

export default DoctorDetails
```

Output

If URL:

```
None
/doctor/2
```

Output:

```
None
Doctor ID: 2
```

Complete App.jsx

```
None
import Navbar from "../components/Navbar"

import Home from "../pages/Home"
import AddDoctor from "../pages/AddDoctor"
import DoctorDetails from "../pages/DoctorDetails"

import {
  Routes,
  Route
} from "react-router-dom"

function App() {
```



```
return (  
  <div>  
    <Navbar />  
    <Routes>  
      <Route  
        path="/"   
        element={<Home />}  
      />  
      <Route  
        path="/add-doctor"  
        element={<AddDoctor />}  
      />  
      <Route  
        path="/doctor/:id"  
        element={<DoctorDetails />}  
      />  
    </Routes>  
  </div>  
)  
}  
  
export default App
```

Folder Structure

```
Shell  
src  
├── components  
│   ├── Navbar.jsx  
│   └── DoctorCard.jsx  
├── pages  
│   └── Home.jsx
```

```
|      |— AddDoctor.jsx
|      |— DoctorDetails.jsx
|— App.jsx
|— App.css
```

Basic Styling

App.css

CSS

```
.navbar{
  display:flex;
  justify-content:space-between;
  align-items:center;
  padding:20px;
  background:black;
  color:white;
}

.container{
  padding:20px;
}

button{
  padding:10px 20px;
  cursor:pointer;
}
```

What Students Learned Today

Students completed:

- React Router setup
- Routes
- Navigation
- Dynamic routing
- `useNavigate`
- `useParams`
- Doctor Details Page

And improved:

None

Doctors App

Day 23 – APIs and GET Requests (Doctors App)

Topics Covered

1. REST API Introduction
 2. HTTP Methods
 3. Axios Setup
 4. GET Request
 5. Handling Response
 6. map() with API Data
 7. Props Usage
-

1. What is REST API?

REST API helps:

- Frontend communicate with backend
-

Real-Time Understanding

Frontend:

- React Application

Backend:

- Server + Database

REST API acts like:

- Middleman
-

Example Flow

None

React App



API Request



Backend Server

↓
Database
↓
Data Response

API We Are Using

None
<https://doc-back.onrender.com/doctors>

Sample API Data

```
JSON
{
  "name": "Teja",
  "salary": "7000000",
  "gender": "Male",
  "age": "26",
  "specialization": "Muscles",
  "id": 2
}
```

2. HTTP Methods

HTTP Methods are used to:

- Perform operations on server
-

Important Methods

Method	Purpose
GET	Fetch data
POST	Add data
PUT	Update data
DELETE	Delete data

Today Focus

Today we will use:

None

GET

GET Method

GET is used to:

- Fetch data from server

Example:

- Get all doctors
-

3. Axios Setup

What is Axios?

Axios is a library used for:

- API requests

Why Axios?

- Easy syntax
 - Better readability
 - Mostly used in React projects
-

Install Axios

Open terminal:

Shell

```
npm install axios
```

Import Axios

Home.jsx

None

```
import axios from "axios"
```

4. GET Request

Now we will:

- Fetch doctors from API
-

Create Doctors State

Home.jsx

None

```
const [doctors, setDoctors] = useState([])
```

Why Array?

Because API returns:

- Multiple doctors
-

Create API Function

None

```
const getDoctors = async () => {  
  
  let response = await axios.get(  
    "https://doc-back.onrender.com/doctors"  
  )  
  
  console.log(response)  
  
}
```

Understanding async/await

API calls take time.

So we use:

- `async`
 - `await`
-

Understanding `axios.get()`

None

```
axios.get()
```

Used for:

- GET request
-

Understanding Response

None

```
response
```

Contains:

- API response object
-

Actual Data

None

```
response.data
```

Contains:

- Doctors array
-

Update State

None

```
setDoctors(response.data)
```

Complete Function

None

```
const getDoctors = async () => {  
  
  let response = await axios.get(  
    "https://doc-back.onrender.com/doctors"  
  )  
  
  console.log(response.data)  
  
  setDoctors(response.data)  
}
```

5. useEffect with API

Now we want:

- Fetch doctors automatically after page load

For this we use:

- useEffect
-

Import useEffect

None

```
import { useEffect } from "react"
```

useEffect Example

None

```
useEffect(() => {  
  
  getDoctors()  
  
}, [])
```

Understanding []

None

```
[]
```

Means:

- Run only once
 - After component load
-

Real-Time Flow

None

Home Page Opens



useEffect Runs



API Request Sent



Data Received



Doctors Display

6. map() with API Data

Now doctors state contains:

- API doctors data

We will display using:

- map()
-

Display Doctors

None

```
{
  doctors.map((doctor) => (
    <DoctorCard
      key={doctor.id}
      id={doctor.id}
      name={doctor.name}
      age={doctor.age}
      gender={doctor.gender}
      specialization={doctor.specialization}
      salary={doctor.salary}
    />
  ))
}
```

```
    ))  
  }  
}
```

Why key?

React needs:

- Unique identification

Best practice:

- Use database id
-

7. Props Usage

Now we pass API data:

- Parent → Child
-

Parent Component
Home.jsx

```
None  
  
<DoctorCard  
  name={doctor.name}  
  specialization={doctor.specialization}  
/>
```

Child Component
DoctorCard.jsx

```
None  
  
function DoctorCard({  
  name,  
  specialization  
) {  
  
  return (  

```

```
    <div>

      <h2>{name}</h2>

      <p>{specialization}</p>

    </div>
  )
}
```

Complete Home.jsx

```
None
import { useEffect, useState } from "react"

import axios from "axios"

import DoctorCard from "../components/DoctorCard"

function Home() {

  const [doctors, setDoctors] = useState([])

  const getDoctors = async () => {

    let response = await axios.get(
      "https://doc-back.onrender.com/doctors"
    )

    console.log(response.data)

    setDoctors(response.data)
  }

  useEffect(() => {

    getDoctors()

  }, [])

  return (
```

```

<div className="container">

  <h1>All Doctors</h1>

  <div className="doctors-container">

    {
      doctors.map((doctor) => (

        <DoctorCard
          key={doctor.id}
          id={doctor.id}
          name={doctor.name}
          age={doctor.age}
          gender={doctor.gender}
          specialization={doctor.specialization}
          salary={doctor.salary}
        />

      ))
    }

  </div>

</div>
)
}

export default Home

```

DoctorCard.jsx

```

None

import { useNavigate } from "react-router-dom"

function DoctorCard({
  id,
  name,
  age,
  gender,
  specialization,
  salary
}) {

```

```

const navigate = useNavigate()

return (
  <div className="card">
    

    <h2>{name}</h2>

    <p>{specialization}</p>

    <p>Age: {age}</p>

    <p>Gender: {gender}</p>

    <p>Salary: {salary}</p>

    <button
      onClick={() => navigate(`/doctor/${id}`)}
    >
      View Details
    </button>

  </div>
)
}

export default DoctorCard

```

Basic Styling

App.css

```

CSS

.container{
  padding:20px;
}

```

```
.doctors-container{
  display:flex;
  gap:20px;
  flex-wrap:wrap;
}

.card{
  width:250px;
  border:1px solid gray;
  padding:20px;
  background:white;
}
```

Folder Structure

```
Shell
src
├── components
│   ├── Navbar.jsx
│   └── DoctorCard.jsx
├── pages
│   ├── Home.jsx
│   ├── AddDoctor.jsx
│   └── DoctorDetails.jsx
├── App.jsx
└── App.css
```

What Students Learned Today

Students completed:

- REST API understanding
- HTTP methods
- Axios setup
- GET request
- API response handling
- map() with API data
- Props usage

And connected:

None

React Frontend + Backend API

Day 24 – POST, PUT, DELETE Operations (Doctors App)

Topics Covered

1. POST Request
 2. PUT Request
 3. DELETE Request
 4. Form Submission to API
 5. Updating UI Dynamically
-

Revision – HTTP Methods

Method	Purpose
--------	---------

GET	Fetch data
-----	------------

POST	Add data
------	----------

PUT	Update data
-----	-------------

DELETE	Delete data
--------	-------------

Today Focus

Today we will perform:

- CRUD Operations
-

What is CRUD?

Operation	Meaning
-----------	---------

Create	Add data
--------	----------

Read	Fetch data
------	------------

Update	Edit data
--------	-----------

Delete	Remove data
--------	-------------

API Endpoint

None

<https://doc-back.onrender.com/doctors>

1. POST Request

What is POST?

POST is used to:

- Add new data to server

Example:

- Add new doctor
-

Real-Time Flow

None

User Fills Form

↓

Submit Form

↓

POST Request

↓

Data Stored in Backend

AddDoctor.jsx

Import Axios

None

```
import axios from "axios"
```

Create Form States

None

```
const [name, setName] = useState("")
const [age, setAge] = useState("")
const [gender, setGender] = useState("")
const [specialization, setSpecialization] = useState("")
const [salary, setSalary] = useState("")
```

Create handleSubmit Function

None

```
const handleSubmit = async (e) => {  
  
  e.preventDefault()  
  
  const newDoctor = {  
    name,  
    age,  
    gender,  
    specialization,  
    salary  
  }  
  
  await axios.post(  
    "https://doc-back.onrender.com/doctors",  
    newDoctor  
  )  
  
  alert("Doctor Added Successfully")  
}
```

Understanding axios.post()

None

```
axios.post(url, data)
```

Understanding Parameters

Parameter	Meaning
url	API endpoint
data	Data to send

Connect Form

None

```
<form onSubmit={handleSubmit}>
```

Complete AddDoctor.jsx

None

```
import { useState } from "react"

import axios from "axios"

function AddDoctor() {

  const [name, setName] = useState("")
  const [age, setAge] = useState("")
  const [gender, setGender] = useState("")
  const [specialization, setSpecialization] = useState("")
  const [salary, setSalary] = useState("")

  const handleSubmit = async (e) => {

    e.preventDefault()

    const newDoctor = {
      name,
      age,
      gender,
      specialization,
      salary
    }

    await axios.post(
      "https://doc-back.onrender.com/doctors",
      newDoctor
    )

    alert("Doctor Added Successfully")

    setName("")
    setAge("")
    setGender("")
    setSpecialization("")
    setSalary("")
  }

  return (

    <div className="container">

      <h1>Add Doctor</h1>
```

```
<form onSubmit={handleSubmit}>

  <input
    type="text"
    placeholder="Enter Doctor Name"
    value={name}
    onChange={(e)=>setName(e.target.value)}
  />

  <br /><br />

  <input
    type="number"
    placeholder="Enter Age"
    value={age}
    onChange={(e)=>setAge(e.target.value)}
  />

  <br /><br />

  <select
    value={gender}
    onChange={(e)=>setGender(e.target.value)}
  >

    <option value="">
      Select Gender
    </option>

    <option value="Male">
      Male
    </option>

    <option value="Female">
      Female
    </option>

  </select>

  <br /><br />

  <input
    type="text"
    placeholder="Enter Specialization"
    value={specialization}
```

```

        onChange={(e)=>setSpecialization(e.target.value)}
      />

      <br /><br />

      <input
        type="number"
        placeholder="Enter Salary"
        value={salary}
        onChange={(e)=>setSalary(e.target.value)}
      />

      <br /><br />

      <button type="submit">
        Add Doctor
      </button>

    </form>

  </div>
)
}

export default AddDoctor

```

2. DELETE Request

What is DELETE?

DELETE is used to:

- Remove data from server

Example:

- Delete doctor

Real-Time Flow

None

Click Delete



DELETE Request

↓
Doctor Removed from Backend

Create deleteDoctor Function
Home.jsx

```
None
const deleteDoctor = async (id) => {

  await axios.delete(
    `https://doc-back.onrender.com/doctors/${id}`
  )

  getDoctors()
}
```

Understanding URL

```
None
/doctors/${id}
```

Example:

```
None
/doctors/2
```

Deletes doctor with ID:

- 2
-

Pass deleteDoctor Function
Home.jsx

```
None
<DoctorCard
  key={doctor.id}
  id={doctor.id}
  name={doctor.name}
```

```
    age={doctor.age}
    gender={doctor.gender}
    specialization={doctor.specialization}
    salary={doctor.salary}
    deleteDoctor={deleteDoctor}
  />
```

Add Delete Button DoctorCard.jsx

```
None
<button
  onClick={() => deleteDoctor(id)}
>
  Delete
</button>
```

Complete Delete Example DoctorCard.jsx

```
None
function DoctorCard({
  id,
  name,
  age,
  gender,
  specialization,
  salary,
  deleteDoctor
}) {

  return (

    <div className="card">

      <h2>{name}</h2>

      <p>{specialization}</p>

      <p>Age: {age}</p>
```



```
    <p>Gender: {gender}</p>

    <p>Salary: {salary}</p>

    <button>
      Edit
    </button>

    <button
      onClick={() => deleteDoctor(id)}
    >
      Delete
    </button>

  </div>
)
}

export default DoctorCard
```

Why getDoctors() Again?

After delete:

- UI should refresh

So we fetch latest data again.

3. PUT Request

What is PUT?

PUT is used to:

- Update existing data

Example:

- Update doctor information
-

Real-Time Flow

None

Click Edit



Update Data



PUT Request



Backend Updated

Simple PUT Example Home.jsx

None

```
const updateDoctor = async (id) => {  
  
  const updatedDoctor = {  
    name: "Updated Doctor",  
    age: 30,  
    gender: "Male",  
    specialization: "Heart",  
    salary: 9000000  
  }  
  
  await axios.put(  
    `https://doc-back.onrender.com/doctors/${id}`,  
    updatedDoctor  
  )  
  
  getDoctors()  
}
```

Understanding axios.put()

None

```
axios.put(url, updatedData)
```

Pass update Function

None

```
<DoctorCard  
  updateDoctor={updateDoctor}
```

```
/>
```

Add Edit Button DoctorCard.jsx

None

```
<button  
  onClick={() => updateDoctor(id)}  
>  
  Edit  
</button>
```

Updated DoctorCard.jsx

None

```
function DoctorCard({  
  id,  
  name,  
  age,  
  gender,  
  specialization,  
  salary,  
  deleteDoctor,  
  updateDoctor  
) {  
  
  return (  
  
    <div className="card">  
  
      <h2>{name}</h2>  
  
      <p>{specialization}</p>  
  
      <p>Age: {age}</p>  
  
      <p>Gender: {gender}</p>  
  
      <p>Salary: {salary}</p>  
  
      <button  
        onClick={() => updateDoctor(id)}  
      >  
        Edit  
      </button>  
    </div>  
  )  
}
```

```
        >
          Edit
        </button>

        <button
          onClick={() => deleteDoctor(id)}
        >
          Delete
        </button>

      </div>
    )
  }

  export default DoctorCard
```

4. Updating UI Dynamically Problem

After:

- POST
- PUT
- DELETE

UI should refresh automatically.

Solution

Call:

```
None
getDoctors()
```

after every operation.

Example

```
None
await axios.delete(url)
```

```
getDoctors()
```

Complete Home.jsx

None

```
import { useEffect, useState } from "react"

import axios from "axios"

import DoctorCard from "../components/DoctorCard"

function Home() {

  const [doctors, setDoctors] = useState([])

  const getDoctors = async () => {

    let response = await axios.get(
      "https://doc-back.onrender.com/doctors"
    )

    setDoctors(response.data)
  }

  useEffect(() => {

    getDoctors()

  }, [])

  const deleteDoctor = async (id) => {

    await axios.delete(
      `https://doc-back.onrender.com/doctors/${id}`
    )

    getDoctors()
  }

  const updateDoctor = async (id) => {

    const updatedDoctor = {
```

```

        name:"Updated Doctor",
        age:30,
        gender:"Male",
        specialization:"Heart",
        salary:9000000
    }

    await axios.put(
        `https://doc-back.onrender.com/doctors/${id}`,
        updatedDoctor
    )

    getDoctors()
}

return (

    <div className="container">

        <h1>All Doctors</h1>

        <div className="doctors-container">

            {
                doctors.map((doctor) => (

                    <DoctorCard
                        key={doctor.id}
                        id={doctor.id}
                        name={doctor.name}
                        age={doctor.age}
                        gender={doctor.gender}
                        specialization={doctor.specialization}
                        salary={doctor.salary}
                        deleteDoctor={deleteDoctor}
                        updateDoctor={updateDoctor}
                    />

                ))
            }

        </div>

    </div>

)

```

```
}
```

```
export default Home
```

Basic Styling

App.css

CSS

```
.container{
  padding:20px;
}

.doctors-container{
  display:flex;
  gap:20px;
  flex-wrap:wrap;
}

.card{
  width:250px;
  border:1px solid gray;
  padding:20px;
  background:white;
}

button{
  padding:10px 20px;
  margin-right:10px;
  cursor:pointer;
}
```

What Students Learned Today

Students completed:

- POST request
- PUT request
- DELETE request
- Form submission to API
- Dynamic UI updates
- CRUD operations

And connected full CRUD with:

None

Doctors App

Day 25 – Advanced React Concepts (Doctors App)

Topics Covered

1. Protected Routes
 2. useMemo
 3. Error Handling
 4. Custom Hooks
 5. Local Storage
-

1. Protected Routes

What are Protected Routes?

Protected Routes are used to:

- Restrict pages
 - Allow only logged-in users
-

Real-Time Example

Without login:

- User should NOT access:
 - Add Doctor Page
 - Doctor Details Page
-

Flow

None

User Logged In ?

↓

YES → Open Page

NO → Redirect to Login

Create Login State
App.jsx

None

```
const isLogin = true
```

Protected Route Example

None

```
<Route
  path="/add-doctor"

  element={
    isLogin

    ?

    <AddDoctor />

    :

    <h1>Please Login First</h1>
  }
/>
```

Understanding Ternary

None

```
condition ? true : false
```

Real-Time Understanding

If user logged in:

- Open Add Doctor Page

Else:

- Show login message
-

Better Structure – ProtectedRoute Component
components/ProtectedRoute.jsx

None

```
function ProtectedRoute({
  isLogin,
  children
}) {

  if(isLogin){

    return children
  }

  return <h1>Please Login First</h1>
}

export default ProtectedRoute
```

Use ProtectedRoute
App.jsx

None

```
<Route
  path="/add-doctor"

  element={

    <ProtectedRoute isLogin={true}>

      <AddDoctor />

    </ProtectedRoute>

  }
/>
```

Understanding children

None

children

Represents:

- Component inside ProtectedRoute

2. useMemo Hook

What is useMemo?

useMemo is used for:

- Performance optimization
-

Why useMemo?

Suppose:

- Large calculations
- Large filtering
- Heavy operations

Without useMemo:

- Recalculates every render

Problem:

- Slow performance
-

Real-Time Example

Doctors filtering:

- Hundreds of doctors

Search becomes slow.

Solution:

- useMemo
-

Import useMemo

None

```
import { useMemo } from "react"
```

Example Without useMemo

None

```
const filteredDoctors = doctors.filter((doctor) => {  
  
  return doctor.name  
    .toLowerCase()  
    .includes(search.toLowerCase())  
  
})
```

Runs every render.

Example With useMemo

None

```
const filteredDoctors = useMemo(() => {  
  
  return doctors.filter((doctor) => {  
  
    return doctor.name  
      .toLowerCase()  
      .includes(search.toLowerCase())  
  
  })  
  
}, [search, doctors])
```

Understanding Dependencies

None

```
[search, doctors]
```

useMemo recalculates only when:

- search changes
 - doctors changes
-

Benefits

- Better performance
- Faster rendering
- Avoid unnecessary calculations

3. Error Handling

Why Error Handling?

API requests may fail because:

- Internet issue
- Server issue
- Wrong API URL

Applications should NOT crash.

try-catch

Used for:

- Handling errors safely
-

Example

None

```
try {  
  
}  
catch(error){  
  
}
```

Real-Time Example

Home.jsx

None

```
const getDoctors = async () => {  
  
  try {  
  
    let response = await axios.get(  
      "https://doc-back.onrender.com/doctors"  
    )  
  
    setDoctors(response.data)  
  
  }  
}
```

```
}  
  
catch(error){  
    console.log(error)  
    alert("Something Went Wrong")  
}  
}
```

Understanding catch(error)

If API fails:

- Error enters catch block
-

Benefits

- Application does not crash
 - User gets proper message
-

4. Custom Hooks

What are Custom Hooks?

Custom Hooks are:

- Reusable React logic
-

Why Custom Hooks?

Suppose:

- Same logic repeated multiple times

Instead of copying:

- Create reusable hook
-

Real-Time Example

Suppose:

- Multiple pages need counter logic

Create:

- useCounter hook
-

Rules

Custom Hook name must start with:

None

use

Example:

- useCounter
 - useDoctors
-

Create Hooks Folder

Shell

src/hooks

Create useCounter.js
hooks/useCounter.js

None

```
import { useState } from "react"

function useCounter() {

  const [count, setCount] = useState(0)

  const increment = () => {

    setCount(count + 1)
  }
}
```



```
const decrement = () => {  
  setCount(count - 1)  
}  
  
return {  
  count,  
  increment,  
  decrement  
}  
}  
  
export default useCounter
```

Use Custom Hook

Home.jsx

```
None  
import useCounter from "../hooks/useCounter"  
  
function Home() {  
  
  const {  
    count,  
    increment,  
    decrement  
  } = useCounter()  
  
  return (  
  
    <div>  
  
      <h1>{count}</h1>  
  
      <button onClick={increment}>  
        Increment  
      </button>  
  
      <button onClick={decrement}>  
        Decrement  
      </button>  
  
    </div>
```

```
)  
}
```

Benefits

- Reusable logic
 - Clean code
 - Better project structure
-

5. Local Storage

What is Local Storage?

Local Storage stores data:

- Inside browser

Data remains even after:

- Refresh
-

Real-Time Example

Store:

- Login status
 - User data
 - Theme settings
-

Store Data

None

```
localStorage.setItem(  
  "isLogin",  
  true  
)
```

Get Data

None

```
localStorage.getItem("isLogin")
```

Remove Data

None

```
localStorage.removeItem("isLogin")
```

Login Example Login Button

None

```
<button onClick={handleLogin}>  
  Login  
</button>
```

handleLogin Function

None

```
const handleLogin = () => {  
  
  localStorage.setItem(  
    "isLogin",  
    true  
  )  
  
  alert("Login Success")  
}
```

Check Login Status App.jsx

None

```
const isLogin =  
  localStorage.getItem("isLogin")
```

Protected Route with Local Storage

None

```
<Route
  path="/add-doctor"

  element={

    isLogin

    ?

    <AddDoctor />

    :

    <h1>Please Login First</h1>
  }
/>
```

Logout Example

None

```
const logout = () => {

  localStorage.removeItem("isLogin")
}
```

Folder Structure

Shell

```
src
├── components
│   ├── Navbar.jsx
│   ├── DoctorCard.jsx
│   └── ProtectedRoute.jsx
├── hooks
│   └── useCounter.js
├── pages
│   ├── Home.jsx
│   ├── AddDoctor.jsx
│   └── DoctorDetails.jsx
```

|
|— App.jsx

What Students Learned Today

Students completed:

- Protected Routes
- useMemo
- Error Handling
- Custom Hooks
- Local Storage

And improved:

None

Doctors App

Day 26 – Context API and React Revision (Doctors App)

Topics Covered

1. Context API
 2. useContext
 3. Provider
 4. React Revision
-

Why Context API?

Suppose:

- Login data needed in:
 - Navbar
 - Home
 - Doctor Details
 - Add Doctor

Without Context API:

- We pass props again and again

Problem:

- Props drilling
-

What is Props Drilling?

Passing props through:

- Multiple components
-

Real-Time Example

None

App

↓

Navbar

↓

Button



Text

Passing same data everywhere:

- Difficult
 - Confusing
-

Solution

None

Context API

What is Context API?

Context API is used for:

- Global state management
-

Real-Time Examples

Store globally:

- Login status
 - Theme
 - User data
 - Language
-

Final Flow Today

None

Context Provider



All Components Access Data

1. Create Context Folder

Folder Structure

```
Shell
src
|
└─ context
```

Create Context File

context/UserContext.jsx

```
None
import { createContext } from "react"

const UserContext = createContext()

export default UserContext
```

What is createContext?

Used to:

- Create global context
-

2. Create Provider

Update UserContext.jsx

```
None
import {
  createContext,
  useState
} from "react"

const UserContext = createContext()

function UserProvider({ children }) {
  const [isLogin, setIsLogin] = useState(false)

  return (
    <UserContext.Provider
      value={{
```



```

        isLogin,
        setIsLogin
      }}
    >

    {children}

  </UserContext.Provider>
)
}

export {
  UserProvider
}

export default UserContext

```

Understanding Provider

Provider shares data:

- To entire application
-

Understanding value

None

```

value={{
  isLogin,
  setIsLogin
}}

```

Global data accessible everywhere.

Understanding children

None

```

children

```

Represents:

- Entire application
-

3. Wrap Application with Provider

main.jsx

None

```
import React from 'react'

import ReactDOM from 'react-dom/client'

import App from './App'

import './index.css'

import { BrowserRouter } from "react-router-dom"

import {
  UserProvider
} from "../context/UserContext"

ReactDOM.createRoot(
  document.getElementById('root')
).render(

  <BrowserRouter>

    <UserProvider>

      <App />

    </UserProvider>

  </BrowserRouter>

)
```

Flow

None

UserProvider

↓

App

↓

All Components

Now all components can access:

- isLogin
 - setIsLogin
-

4. useContext Hook

What is useContext?

useContext is used to:

- Access context data
-

Import useContext

None

```
import { useContext } from "react"
```

Access Context Data

Navbar.jsx

None

```
import {
  useContext
} from "react"

import UserContext from "../context/UserContext"

function Navbar() {

  const {
    isLogin,
    setIsLogin
  } = useContext(UserContext)

  return (

    <div>
```

```
    <h1>Doctors App</h1>

    <h2>{isLoggedIn ? "Logged In" : "Logged Out"}</h2>

  </div>
)
}

export default Navbar
```

Understanding useContext

```
None
useContext(UserContext)
```

Gets:

- Global data
-

Output

If:

```
None
isLoggedIn = false
```

Output:

```
None
Logged Out
```

5. Login and Logout Add Buttons

```
None
<button
  onClick={() => setIsLogin(true)}
>
```

```
      Login
    </button>

    <button
      onClick={() => setIsLogin(false)}
    >
      Logout
    </button>
```

Updated Navbar.jsx

```
None
import {
  useContext
} from "react"

import UserContext from "../context/UserContext"

function Navbar() {

  const {
    isLogin,
    setIsLogin
  } = useContext(UserContext)

  return (

    <div className="navbar">

      <h1>Doctors App</h1>

      <div>

        <h3>
          {
            isLogin

              ?

              "Logged In"

            :


```

```

        "Logged Out"
      }
    </h3>

    <button
      onClick={() => setIsLogin(true)}
    >
      Login
    </button>

    <button
      onClick={() => setIsLogin(false)}
    >
      Logout
    </button>

  </div>

</div>
)
}

export default Navbar

```

Real-Time Understanding

```

None
Click Login
  ↓
Context State Updates
  ↓
All Components Receive Updated Data

```

Use Context in ProtectedRoute ProtectedRoute.jsx

```

None

import {
  useContext
} from "react"

import UserContext from "../context/UserContext"

```

```
function ProtectedRoute({
  children
}) {

  const {
    isLogin
  } = useContext(UserContext)

  if(isLogin){

    return children
  }

  return <h1>Please Login First</h1>
}

export default ProtectedRoute
```

Benefit

Now:

- No need to pass login props manually

React Revision Before Project

React Concepts Learned

Day	Topics
Day 18	React Intro, JSX, Components
Day 19	Props, useState, Forms
Day 20	map(), Conditional Rendering
Day 21	filter(), Search, Deployment
Day 22	Routing, useNavigate, useParams
Day 23	APIs, GET Request, Axios

Day 24 POST, PUT, DELETE

Day 25 Protected Routes, useMemo,
Hooks

Important React Hooks Revision

Hook	Purpose
useState	Manage state
useEffect	Side effects
useNavigate	Navigation
useParams	Route parameters
useMemo	Performance
useContext	Access context

Important JavaScript Methods Revision

Method	Purpose
map()	Loop arrays
filter()	Filter data
includes()	Check text
toLowerCase()	Convert lowercase

Project Concepts Completed

Students already know:

- CRUD Operations
 - Routing
 - API Integration
 - Search Functionality
 - Protected Routes
 - Forms
 - Dynamic Rendering
-

Final Doctors App Features

None

- ✓ Add Doctor
 - ✓ View Doctors
 - ✓ Update Doctor
 - ✓ Delete Doctor
 - ✓ Search Doctors
 - ✓ Filter Doctors
 - ✓ Routing
 - ✓ API Integration
 - ✓ Protected Routes
 - ✓ Context API
-

Folder Structure

Shell

```
src
├── components
│   ├── Navbar.jsx
│   ├── DoctorCard.jsx
│   └── ProtectedRoute.jsx
├── context
│   └── UserContext.jsx
├── hooks
│   └── useCounter.js
├── pages
│   ├── Home.jsx
│   ├── AddDoctor.jsx
│   └── DoctorDetails.jsx
└── App.jsx
```

Basic Styling

App.css

CSS

```
.navbar{
  display: flex;
  justify-content: space-between;
  align-items: center;
```

```
padding:20px;
background:black;
color:white;
}

button{
padding:10px 20px;
margin-left:10px;
cursor:pointer;
}
```

What Students Learned Today

Students completed:

- Context API
- useContext
- Provider
- Global state management
- React revision

And completed preparation for:

None

Major React Project
